Problem Set 2: Language Models

Abraham Sharum

Due Date: September 30, 2024
Class: CS 4343 - Natural Language Processing

October 1, 2024

Language Models

Data Structures

```
The used data structures consisted of:
Dictionaries
Lists
```

Implementations:

Dictionaries: The dictionaries were used to create nested dictionaries in order to store all potential trigrams such as [This: {the: {guy: 20, girl:10}} {Where: {we: 50, I: 20}}] and potential bigrams as [This: {is :20}, {will:30} We: {are: 50} , {will : 20}].

Lists: The lists were used when a word/sentence needs to be split, when aggregation needs to occur, or for getting all words from a document. Most implementations of lists were for trivial tasks.

Algorithms and Space/Time Complexity Analysis:

main.py: Contains the following methods, time and space complexity are both O(n):

1. Doc of size n loaded into memory $\theta(n)$ space and time.
2. Doc forced lowercase $\theta(n)$ time.
3. Doc split $\theta O(n)$ time.
4. Cache/load 3 dictionaries, the size of the three dictionaries being V, V^2, and V^3 for unigrams, bigrams, and trigrams respectively. While the 2 nested dictionaries potentially have a size of $3V$, where V is the lexicon size. Each document is looped through to serialize them to json. This makes runtime and space $6V^3$ (Note: Assuming that the used document is not full of entirely distinct words, $V < n$. This assumption is to be maintained throughout the document for consistent asymptotic complexity analysis). Removing constants the result is $O(V^3)$ time and space.
5. With a runtime of $3n+6V^3$ and $n+6V^3$ of stored space for the initialization. Given the three methods in main.py that each contain $O(V)$ runtime and space, the total runtime is $3n+6V^3$ and the total space used is $n+6V^3$. Thus making the time and space complexities of main $O(n+V)$, despite the assumption made before $O(V^3)$ is significantly larger than n, thus the dominant space and time complexity is $O(V^3)$. Therefore, the asymptotic time and space complexity of main.py is $O(V^3)$.

main(message,n_grams): Contains the following methods, time and space complexity are both O(n):

1. **predict_next(input):**

   This method takes the message parameter from main(message,n_grams) as an argument. This message has a size of k.

   Constant loop to add the next 10 words

Each iteration contains a call of the predict() method.

There is also a call of grams.find_input_prob which has a time and space complexity of *O(k)*

Given there are potentially 10 predict() calls (method may end due to try/except clause), in which each call of predict has an asymptotic complexity of *O(V)* the space and time complexity is *10V* making the asymptotic time complexity *O(V+k)*.

2. **predict(words):**

   This method checks for potential trigram and bigram matches via nested dictionaries all operations are constant.

   The time complexity of this method is dominated by the getWord() method call.

   It uses constant space as the nested dictionaries are instance variables.

   The asymptotic time complexity is *O(V)* and the asymptotic space complexity is *O(1)*.

3. **getWord(possible_next):**

   This method uses the max() function to loop through the frequencies of each word in the posible_next dictionary and saves/returns the max.

   Constant space as the only new instance in memory is a string variable.

   Thus, the asymptotic time complexity is *O(V)* and space complexity is *O(1)*.

grams.py: Contains the following methods, time and space complexity are both $O(V^3)$:

1. **tokenize_doc_unis(doc):**

   This method loops through a loaded doc of size n, and creates a dictionary of (key: word) and (value: word frequency).

   Both the space and time complexities of this method are $\theta(n)$ as it loops to n and creates a dictionary of size V.

   Thus, the asymptotic time and space complexity is $\theta(n)$

2. **tokenize_doc_bis(doc):**

   This method loops through a loaded doc of size n, and creates a dictionary of (key: bigram) and (value: bigram frequency).

   Each bigram frequency is smoothed to ensure non-zero probabilities.

   The space complexity of this method is $O(V^2)$ as it loops to n and creates a dictionary of size $O(V^2)$.

   Thus, the asymptotic time complexity is $\theta(n)$ and the asymptotic space complexity is $O(V^2)$

3. tokenize_doc_tris(doc):

   This method loops through a loaded doc of size n, and creates a dictionary of (key: trigram) and (value: trigram frequency).

   The space complexity of this method is $O(V^3)$ as it loops to n and creates a dictionary of size $O(V^3)$.

   Thus, the asymptotic time complexity is $\theta(n)$ and the asymptotic space complexity is $O(V^3)$

4. **def nested(n_gram):**

   This method contains an algorhtim to create nested dictonaries given trigrams/bigrams and their frequencies.

   The nested dictionary generated is at most three dictionaries deep meaning the size is 3V.

   The space complexity is dominated by the loaded in argument n_gram which can be the trigram dictionary which has an asymptotic space complexity of $O(V^3)$.

   The time complexity is $O(V^3)$ as the trigram dictionary is looped through.

   Thus, the asymptotic time and space complexity is $O(V^3)$.

5. **def find_input_prob(sentence,unigrams):**

   This method simply loops through a generated sentence and calculates the probaility of each word within it, and saves it to a dictonary of (key:word) and (value:probability).

   The time complexity is $\theta(k)$ where k is the size of the generated sentence.

   The space complexity is $O(k + V)$ as the lexicon is also loaded via an argument.

   Thus, the asymptotic time complexity is $\theta(k)$ and the asymptotic space complexity is $O(k + V)$.

6. **def save_to_file(to_save,file_name,is_output):**

   This method saves a passed in data structure to a json file given the passed in name.

   The space and time complexities are dominated by the passed in data structure. Given that the trigram dictonary can be passed in the complexities become $O(V^3)$.

   The os.makedirs() call recursively walks a file tree given it is only a step forward and a generation of a file it's runtime is negligible.

   Thus, the asymptotic complexity for both space and time is $O(V^3)$.

bot.py: Contains the following methods, time and space complexity are both $O(V^3)$

1. Most of the code within this file is methods and classes from the discord API which have constant time and space complexities. The runtime of this file is dominated by the import of the main.py as its' time and space complexities are adopted when instantiated. There are a few loops in which they travel to n however none of the loops are nested meaning for k number of loops to n where k $\leq$ 5, 5n is still asymptotically O(n). Thus, the asymptotic time and space complexity of bot.py is $O(V^3)$.

Conclusion

Across multiple tests the trigram model seems to generate slightly more probable sentences than the bigram model. Based on further testing loading the cached dictonaries as opposed to generating them again seems to increase the total speed of the application by roughly a factor of 8. Also due to the nested dictonaries approach, the word generation is near instant as hashing means skipping iteration to find probable trigrams and bigrams.

Deliverable

Command

python3 bot.py Either L to load or S to cache

Output

Output.txt


```
[
  "Output: we are the king henry the king henry the king henry the king",
  "Bigram Probability: P( we are the king henry the king henry the king henry the king) =
      —27.677314196639905",
  "P(we) = 2.4674e—06",
  "P(are) = 3.5062e—06",
  "P(the) = 0.0003280059",
  "P(king) = 1.7688e—06",
  "P(henry) = 4.19e—08"
]
[
  "Output: we are the only son of mine own part i am not i",
  "Trigram Probability: P( we are the only son of mine own part i am not i) =
      —68.70202668862217",
  "P(we) = 2.467448917e—06",
  "P(are) = 3.506243115e—06",
  "P(the) = 0.000328005935212",
  "P(only) = 3.6761852e—08",
  "P(son) = 6.7126456e—08",
  "P(of) = 0.000124866559909",
  "P(mine) = 2.21712855e—07",
  "P(own) = 8.6385781e—08",
  "P(part) = 3.2755977e—08",
  "P(i) = 0.000168160726624",
  "P(am) = 9.33929846e—07",
  "P(not) = 2.305162994e—05"
]
[
  "Output: such that there is the king henry the king henry the king henry",
  "Bigram Probability: P( such that there is the king henry the king henry the king henry) =
      —38.086421534136896",
```

```
  "-P(such)-=-4.741e—07",
  "-P(that)-=-4.03037e—05",
  "-P(there)-=-6.148e—07",
  "-P(is)-=-2.8712e—05",
  "-P(the)-=-0.0003280059",
  "-P(king)-=-1.7688e—06",
  "-P(henry)-=-4.19e—08"
]
[
  "Output:-such-that-there-is-no-more-of-this-world-i-would-not-have",
  "-Trigram-Probability:-P(-such-that-there-is-no-more-of-this-world-i-would-not-have)-=-
      —68.08155958946652",
  "-P(such)-=-4.74071806e—07",
  "-P(that)-=-4.0303650466e—05",
  "-P(there)-=-6.1479515e—07",
  "-P(is)-=-2.8712031008e—05",
  "-P(no)-=-3.392009435e—06",
  "-P(more)-=-1.060468015e—06",
  "-P(of)-=-0.000124866559909",
  "-P(this)-=-1.3192277285e—05",
  "-P(world)-=-1.07848244e—07",
  "-P(i)-=-0.000168160726624",
  "-P(would)-=-1.064736187e—06",
  "-P(not)-=-2.305162994e—05",
  "-P(have)-=-9.316562483e—06"
]
[
  "Output:-i-want-to-the-king-henry-the-king-henry-the-king-henry-the",
  "-Bigram-Probability:-P(-i-want-to-the-king-henry-the-king-henry-the-king-henry-the)-=-
      —33.05077084768223",
  "-P(i)-=-0.0001681607",
  "-P(want)-=-1.5e—09",
  "-P(to)-=-0.0001416822",
  "-P(the)-=-0.0003280059",
  "-P(king)-=-1.7688e—06",
  "-P(henry)-=-4.19e—08"
]
[
  "Output:-i-want-to-send-him-word-you-are-a-fool-and-a-good",
  "-Trigram-Probability:-P(-i-want-to-send-him-word-you-are-a-fool-and-a-good)-=-
      —67.88546478414332",
  "-P(i)-=-0.000168160726624",
  "-P(want)-=-1.536716e—09",
  "-P(to)-=-0.000141682195438",
  "-P(send)-=-6.945659e—09",
  "-P(him)-=-6.976348225e—06",
  "-P(word)-=-3.1429966e—08",
  "-P(you)-=-6.4591508858e—05",
  "-P(are)-=-3.506243115e—06",
  "-P(a)-=-7.5582406311e—05",
  "-P(fool)-=-2.6433333e—08",
```

```
  "P(and) = 0.00030354345703",
  "P(good) = 1.69982865e-06"
]
```

Code

main.py

```
# Abraham Sharum
# CS4343
# PS2
# Due Date: September 30, 2024 11:00PM
import string
import grams
import json
import random

doc = open("./shakespear.txt","r")
doc = doc.read()
doc = doc.translate(str.maketrans('', '',string.punctuation))
doc = doc.lower()
doc = doc.split()

to_save = input(str("Would you to load or save the dictonaries? 'l'\\'s':"))

if to_save.lower() == 's':
    to_save = True
elif to_save.lower() == 'l':
    to_save = False

if (to_save == True):
    print("saving...")
    unigram_lm = grams.tokenize_doc_unis(doc)
    bigram_lm = grams.tokenize_doc_bis(doc,unigram_lm)
    trigram_lm = grams.tokenize_doc_tri(doc)

    nested_bigrams = grams.nested(bigram_lm)
    nested_trigrams = grams.nested(trigram_lm)

    grams.save_to_file( unigram_lm, "unigrams",False)
    grams.save_to_file( bigram_lm, "bigrams",False)
    grams.save_to_file( trigram_lm, "trigrams",False)
    grams.save_to_file( nested_bigrams, "nbigrams",False)
    grams.save_to_file( nested_trigrams, "ntrigrams",False)
    print("Save complete!")

else:
    print("loading....")
    unigram_lm = json.loads(open("./ngrams/unigrams.json").read())
    bigram_lm = json.loads(open("./ngrams/bigrams.json").read())
    trigram_lm = json.loads(open("./ngrams/trigrams.json").read())
    nested_bigrams = json.loads(open("./ngrams/nbigrams.json").read())
    nested_trigrams = json.loads(open("./ngrams/ntrigrams.json").read())
    print("All dictonaries loaded!")
```

```
def main(message,n_gram):

    #predict the output

    def predict_next(input):

        for i in range(10):
            try:
                if i == 0:
                    input += " " + predict(input.split()) + " "
                elif i < 9:
                    input += predict(input.split()) + " "
                else:
                    input += predict(input.split())
            except:
                return input.split()[len(input.split())-1],0

        probs = grams.find_input_prob(input,unigram_lm)
        return input, probs

    def predict(words):
        if len(words) >= 2:
            first_word = words[len(words)-2]
        if len(words) >= 1:
            second_word = words[len(words)-1]


        if(len(words) >= 2 and n_gram == "tri"):


            if first_word in nested_trigrams and second_word in nested_trigrams[first_word]:
                next_word = getWord(nested_trigrams[first_word][second_word])
            if next_word == None and second_word in nested_bigrams:
                next_word = getWord(nested_bigrams[second_word])
            return next_word


        elif(len(words) >= 1):
            if second_word in nested_bigrams:
                next_word = getWord(nested_bigrams[second_word])
                return next_word

    def getWord(possible_next):
            #code using max.choices() for weighted probabilities
            """"
            num_items = len(possible_next.items())
            words = [None] * num_items
            frequncies = [None] * num_items
            i = 0
            total = 0
```

```
            for word, count in possible_next.items():
                words[i] = word
                frequncies[i] = count
                total += count
                if i < num_items:
                    i+=1

            word_probs = [None] * num_items

            i = 0
            for frequency in frequncies:
                word_probs[i] = frequency / total
                if i < num_items:
                    i+=1

            choice = max(random.choices(words,word_probs))
            """
            choice = max(possible_next, key=possible_next.get)

            return choice

    return predict_next(message)
```

bot.py

```
# Abraham Sharum
# CS4343
# PS2
# Due Date: September 30, 2024 11:00PM
import math
import string
import discord
import main

intents = discord.Intents.default()
intents.message_content = True

client = discord.Client(intents=intents)
ashaur02 = "MTI4ODc0MDU3NjQwNTIyOTYwOA.Gn1SJr.YV6—
    X8_tozIty59m6bpFF4hN4zwAZgvmlVlTOA"

@client.event
async def on_ready():
    print("Constructing Model...")
    print(f'We have logged in as {client.user}')
    print("Model constructed! Ready to generate, please navigate to discord.")

@client.event
async def on_message(message):
    content = message.content
    content = content.lower()

    if message.author == client.user:
        return

    if message.content.startswith('$hello'):
        await message.channel.send('Hello!')


    if client.user in message.mentions:
        content = content.split()
        prompt = ""

        for i in range(1,len(content)):
            prompt += " " + content[i]

        program = main

        prompt = prompt.translate(str.maketrans('', '',string.punctuation))

        biresponse = program.main(prompt,"bi")
        biprobs = biresponse[1]

        total = 0
```

```
        if biprobs == 0:
            pass
        else:

            for key in biprobs.keys():
                if biprobs[key] == 0:
                    total = 0
                    break
                total += biprobs[key]
            await message.channel.send("Bigram generation: " + biresponse[0])

            await message.channel.send(f"Bigram Probability: P({biresponse[0]}) = {str(round(
                total,15)):4}")

            biprob = []
            biprob.append("Output:" + biresponse[0])
            biprob.append(f" Bigram Probability: P({biresponse[0]}) = {str(round(total,15)):4}")
            for key in biprobs.keys():
                biprob.append(f" P({key}) = {str(round(math.pow(10,biprobs[key]),10)):4}")
            main.grams.save_to_file(biprob,"bigramOutput",True)

        total = 0
        triresponse = program.main(prompt,"tri")
        if biprobs == 0:
            await message.channel.send(f"Sorry, I have never seen the word {triresponse[0]} before
                ,I can't predict the next words!")
        else:
            await message.channel.send("Trigram generation:"+ triresponse[0])
            triprobs = triresponse[1]
            for key in triprobs.keys():
                if triprobs[key] == 0:
                    total = 0
                    break
                total += triprobs[key]
            await message.channel.send(f"Trigram Probability: P({triresponse[0]}) = {str(round(
                total,15)):4}")
            if total == 0:
                await message.channel.send(f"The sentence proability being zero means there
                    exists a word that is not in the lexicon and was not used for sentence
                    generation.")
            triprob = []
            triprob.append("Output:" + triresponse[0])
            triprob.append(f" Trigram Probability: P({triresponse[0]}) = {str(round(total,15)):4}
                ")
            for key in triprobs.keys():
                triprob.append(f" P({key}) = {str(round(math.pow(10,triprobs[key]),15)):4}")
            main.grams.save_to_file(triprob,"trigramOutput",True)

client.run(ashaur02)
```

grams.py

```
# Abraham Sharum
# CS4343
# PS2
# Due Date: September 30, 2024 11:00PM
import math
import json
import os
def tokenize_doc_unis(doc):
    unigram_lm = dict()
    for token in doc:
        if token in unigram_lm:
            unigram_lm[token] += 1
        else:
            unigram_lm[token] = 1

    return unigram_lm

def tokenize_doc_bis(doc,unigram_lm):
    bigram_lm = dict()

    for i in range(0, len(doc) - 1,):
        bigram = doc[i] + "-" + doc[i+1]

        if bigram in bigram_lm:
            bigram_lm[bigram] += 1
        else:
            bigram_lm[bigram] = 1

    uni = len(unigram_lm)
    total = sum(bigram_lm.values())
    for key in bigram_lm.keys():
        bigram_lm[key] = math.log((bigram_lm[key] + 1)) - math.log((total + uni))

    return bigram_lm

def tokenize_doc_tri(doc):
    trigram_lm = dict()

    for i in range(0, len(doc) - 2,):
        trigram = doc[i] + "-" + doc[i+1] + "-" + doc[i+2]

        if trigram in trigram_lm:
            trigram_lm[trigram] += 1
        else:
            trigram_lm[trigram] = 1


    return trigram_lm
```

```
def nested(n_gram):
    nested_ngrams = dict()


    for key, freq in n_gram.items():
        key = key.split()
        if len(key) > 2:
            if key[0] not in nested_ngrams:
                nested_ngrams[key[0]] = dict()

            if key[1] not in nested_ngrams[key[0]]:
                nested_ngrams[key[0]][key[1]] = dict()
            nested_ngrams[key[0]][key[1]][key[2]] = freq
        elif len(key) > 1:
            if key[0] not in nested_ngrams:
                nested_ngrams[key[0]] = dict()

            nested_ngrams[key[0]][key[1]] = freq

    return nested_ngrams

def find_input_prob(sentence,unigrams):

    probs = dict()
    sentence = sentence.split()

    for word in sentence:
        try:
            probs[word] = math.log(unigrams[word]) - math.log(sum(unigrams.values()))
        except:
            print(f"{word} is not in the training data!")
            probs[word] = 0
    return probs

def save_to_file(to_save,file_name,is_output):
    if is_output:
        parent_dir = "./output/"
    else:
        parent_dir = "./ngrams/"

    try:
        os.makedirs("./ngrams")
    except:
        pass
    try:
        os.makedirs("./output")
    except:
        pass


    if is_output:
```

```
        with open(parent_dir + file_name + ".json", "w") as file:
            json.dump(to_save, file,indent=2)
    else:
        with open(parent_dir + file_name + ".json", "w") as file:
            json.dump(to_save, file)
```